

The Challenge of Assessing and Controlling Complexity in a Large Portfolio of Software Systems

Dag I.K. Sjøberg

Department of Informatics,
University of Oslo, P.O. Box 1080
Blindern, NO-0316 Oslo, Norway

Dag.Sjoberg@ifi.uio.no

Erik Odberg

Telenor Norway, Snarøyveien 30,
NO-1331 Fornebu,
Norway

Erik.Odberg@telenor.com

Borger Warlo

Telenor Norway, Snarøyveien 30,
NO-1331 Fornebu,
Norway

Borger.Warlo@telenor.com

ABSTRACT

Controlling IT complexity in a large organization's portfolio of constantly evolving software systems is crucial in a business world with tighter economic constraints and an increased focus on business agility. Unfortunately, there are no commonly accepted metrics for software complexity, and the few metrics that exist have not been validated as good predictors of the impact on cost in a given context. Telenor Norway has therefore developed its own method to help assess the cost of IT complexity from a broad perspective. For all the systems affected by a given project, the architects and experts of each system subjectively estimate the change in the annual cost of developing and maintaining it as a result of the change in complexity brought about by that project. The method was subjectively evaluated and improved in six pilot studies. Following these experiences, it was decided to gradually introduce the method throughout the whole organization, in the belief that this would allow Telenor Norway to make better architectural decisions, and to achieve a reduced cost of ownership and an increased strategic flexibility. Work continues to improve the method.

Categories and Subject Descriptors

D.2 SOFTWARE ENGINEERING

General Terms

Design, Management

Keywords

Software complexity, software costs, software process improvement

1. INTRODUCTION

Large private enterprises and government agencies generally have a comprehensive portfolio of interdependent information systems. A major challenge faced by these organizations is how to control the increasing complexity and the associated costs of such systems as the portfolio evolves. Typically, the number and size of the systems and the components of the individual systems increases over time, as well as the number of relationships

between these systems and components. The continuous change in the nature of the business and often the increased complexity of the domain lead to more complex system requirements and information models, and the increased functionality of the systems. The greater complexity of the software portfolio leads to a higher cost of developing new systems and maintaining existing ones, a reduced business agility, a longer time-to-market, and an increased dependence on highly skilled people.

In such an environment, there is a continuous need to modernize the system portfolio. However, making decisions about when, what and how to modernize is often difficult due to a lack of (1) knowledge of which modernization decision criteria are important, (2) reliable and practically useful measures of software complexity and other system quality issues, and (3) reliable measures of the potential economic effects of modernization decisions before these modernizations take place [1].

Telenor Norway is part of the Telenor group, which is the fifth largest mobile operator in the world. In Telenor Norway, the economic gains of any project are estimated in the corresponding business case, but it rarely includes estimates of the costs incurred due to the increased complexity caused by the actual project.

More specifically, most of the focus is normally on project costs, i.e., application development costs (primarily capex = capital expenditures (investments), partly opex = operational expenditures). Some of the focus is also on future application maintenance costs (opex). (Note that in this context maintenance does not include new functionality; that is, the focus is on corrective and adaptive maintenance in the classification by Swanson [2].) However, in general, the business case does not include a systematic way of describing cost avoidance (primarily capex) and future project costs due to increased complexity. One of the consequences of this is that the economic gains of running IT modernization projects that involve re-engineering or simplification (e.g. restructuring) of the existing software portfolio are rarely estimated.

The IT Architecture Department felt that it was difficult to control the overall complexity of the portfolio and that projects that involved re-engineering were prioritised less than deserved because the economical impacts of reducing complexity were difficult to include in the business cases. Telenor Norway therefore needed a method that could be used to assess the change in complexity that could be caused by a new project, and the economic impact of this increased or reduced complexity. A better insight into the economic impacts of a change in complexity is useful for project prioritization (at least for marginal projects), for software architecture decisions, as well as for promoting

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.

PROFES, June 21-23, 2010, Limerick, Ireland.

Copyright: ACM

simplification, reuse, and other IT modernization initiatives. This need forms the motivation for the work described in this article.

2. COMPLEXITY MEASURES

Because IT complexity affects cost for all large software systems, we searched the literature for a way of measuring complexity for this purpose. We found a high-level description in which complexity is characterized by “the number and variety of components and their interactions, and their combined rate of change.” [3]. This seemed sensible, but less obvious was the question of what is, and how do we measure, a component, interaction and rate of change in our context? And how do we relate these concepts to costs?

By probing deeper into the software engineering literature, we found a plethora of software attributes (for example, the CK metrics [4]) used in a range of studies to demonstrate relationships between various cost-drivers. However, it is difficult to combine these metrics into one over-arching measure of complexity that can be used to compare different systems or even different versions of the same system [5]. One proposal for assessing the maintainability of complete systems is the Maintainability Index [6]. Although software maintainability is not the same as (or more precisely, the inverse of) software complexity, these properties are closely related. Nevertheless, the use of this index has been reported in several projects, although there is no indication in the literature that this formula is generally applicable.

A general challenge in the practical use of many of the software measures reported in the literature is that they are often strongly interdependent, i.e., for a given system, improving the value of one attribute may imply less favourable values for other attributes. For example, achieving high cohesion and low coupling among modules or classes would generally be easier if one were to increase their size, but then the complexity would increase just because of the increase in the size of the module or class. Few studies assess the trade-offs between the values of such metrics. Darcy and Kemerer state that “the exact relationship between the metrics and prediction of outcomes of managerial interest need to be calibrated for local influences” [7]. While this is certainly true, the question remains of how to apply this calibration in practice. Within Telenor Norway, an organization with 4-500 IT systems and in which large projects may require changes to perhaps 30-40 of these, performing such a calibration would be unfeasible.

While the measures described above operate at the level of one single system, here we are primarily interested in assessing complexity at the portfolio level. Of course, the complexity of the individual systems constitutes much of the complexity of the portfolio, but there is still additional complexity introduced when managing the overall portfolio. Little software engineering research has focused on this issue [8]. In particular, we found no validated ways of how to measure the complexity of software portfolios.

3. COMPLEXITY VS. COST

The primary interest for Telenor Norway is not the complexity of its portfolio per se, but the extent to which it affects costs. Telenor needs a clear idea of the economic consequences of increasing or reducing the complexity of its portfolio. Such information would in turn be important for (1) project prioritization, (2) promoting modernization initiatives, (3) deciding on software architecture solutions, and (4) increasing the awareness of (the cost of)

complexity. As described above, there is no universal, practical way of defining or measuring software complexity. We also found no generally applicable method in the literature that we could use to relate complexity to cost in our context. This is to be expected, because the future costs due to a certain level of complexity will depend on the exact form and context of the software evolution, for example, the types of new systems to be added, the changes to be made to the existing systems, the organisation of new projects, and the competence of the people taking part. As a consequence, we decided to develop a method of our own, tailored to our needs.

4. THE APPROACH AT TELENOR

Given the lack of operational definitions of complexity, and the universal relationship between complexity and cost described in the state-of-the-art literature, we propose a pragmatic, black-box way of describing complexity. Note that we aim to control the complexity and thus the costs, so that neither becomes unmanageably high. We are therefore primarily interested in understanding the extent to which new projects contribute to increasing or reducing complexity rather than obtaining the “absolute value” of the complexity. Furthermore, we are primarily interested in assessing the economical consequences of changes in complexity. We therefore define complexity in our context as follows:

The IT complexity is reflected by those characteristics of the software portfolio that contribute to increased or reduced development and maintenance costs in the future.

The question then arises of how to calculate these costs. We could have started work on developing sophisticated regression models to describe the relationships between product and project characteristics and costs using empirical data from Telenor. However, we had neither the resources nor the time to identify and measure the relevant system and project characteristics on a scale necessary to establish a useful regression model. Another problem with using a regression model is that it only uses data from past projects. Thus the predictive capability of the model is hampered by a context that is continuously changing, and its reliability will be governed by the extent to which new projects are different from past ones. Consequently, we developed a method using human judgement to estimate the economic consequences of implementing a given project on the software portfolio.

5. STEPS OF THE TELENOR METHOD

The method consists of four steps for determining the economic impact of a given project. Table 1 shows these steps, together with an example from a restructuring project. The last row shows that the annual cost gain of running that project is estimated to be 290 kEuro. (A minus “-” indicates cost saving.) The numbers are real, but the system names are made up for reasons of confidentiality. The steps of the method are as follows.

Step 1: Identify the Affected Systems. The purpose of the first step is to identify all the IT systems that will be affected by the project. This identification is carried out by the architects, by inspecting the top level of the relevant architecture documents and is carried out as a normal part of the analysis phase in all projects.

Step 2: Extract the Affected Cost Base. Determine the affected cost base for each of the affected systems. The cost base is an estimate of those annual costs associated with a given system that

Table 1. Basic steps of the method and example from a restructuring project.

Step 1: Identify affected systems	Step 2: Determine cost base per system	Step 3: Estimate complexity change per system		Step 4: Determine total economic impact	
From project top level design	Based on previous investments, costs and future strategic economical guidance	Based on experience, gut feeling, check lists and guidelines		Added in business case as cost (+) or gain (-)	
TST	10 MEuro	X	-3 %	=	-300 kEuro
Helitor Cotex	0.5 MEuro	X	0.5 %	=	2.5 kEuro
Vision II	1.0 MEuro	X	0.5 %	=	5.0 kEuro
Futurena	0.5 MEuro	X	0.5 %	=	2.5 kEuro
Mikro	0.5 MEuro	X	0 %	=	0
Copic	1.0 MEuro	X	0 %	=	
Annual change of development and maintenance cost after implementation of project					-290 kEuro

will be affected by the increase in complexity, including all project costs (capex, opex)) and application maintenance costs (opex). Operating costs (opex) are not part of the affected cost base, mainly because any impact on these costs is handled via existing business case evaluations. In addition to the fact that there is already a procedure in place to deal with opex, it is to a lesser extent related to software complexity than the other costs.

The estimate is made mainly by using historical data from the last three years, that is, actual costs. In addition, the estimate should be adjusted by taking into account any expected future changes to the system that could affect the historical cost base. Note that the affected cost base for each system is not determined by the project, but is settled by the Finance Department and IT Architecture Department annually, and approved by the IT Management Team.

Step 3: Estimate the Change of Complexity Per System. The system experts assess the complexity or cost change per system caused by the new project on the basis of experience and gut feeling. Checklists and guidelines are also consulted and will be developed further as the method is more widely used. Furthermore, relevant historical data will be provided, for example, on the extent to which different types of change contributed to changes of complexity in former projects.

The subjective estimation of the change is carried out by a group of architects and systems experts, as opposed to individuals. Group decisions are widely used in software evolution decision-making, both in private and public organizations [9]. Group decisions led to better estimates than mechanical averaging of individual estimates in an experiment on software effort estimation [10].

For each system that is possibly affected, the project manager and the experts on the given system meet in a two-hour session during which they perform the estimation. The values and comments are filled in online in a spreadsheet that is displayed via a projector. Both future application development and maintenance costs are estimated. Table 1 shows a simplified version of the spreadsheet. The real spreadsheet contains, for each system, one row for the expected increase and one for the expected decrease.

The project manager organizes the meeting and provides all the relevant material. The project architect makes notes during the meeting and is responsible for reporting the outcome.

The increase or reduction in costs for each system due to the project under consideration may be estimated as a percentage of the current cost base or in absolute terms. The experts select the easier of these for the given case. It may sometimes be easier to indicate that a given project will result in a cost saving of (say) 5%, while on another project it could be said that (say) a further two person years are required compared with the old system. In practice, it is possible to switch between the two approaches when estimating the complexity change for the same project. In any case, the change in complexity is measured in terms of its economic consequences.

One of the columns is used for comments. For example, one of the systems had a comment in the “increase” row that some increase in complexity could be expected due to the increased testing required on a module for improving security. The “decrease” column for the same system contained the comments that (1) it would be easier to provide the same services in the future (about a 2-3 person-month reduction); (2) the software would be cleaner, better structured, more comprehensible and more robust, and the tests would be better (about a 1 person-year reduction in application development and maintenance); (3) new methods would make the system faster and provide a greater range of options due to the use of web-services.

For some small systems, the cost base may be negligible. For some (other) small projects or systems, it may be obvious that the change in complexity caused by the project is also negligible. In both cases, one can quickly conclude that the economic impact is zero.

Step 4: Total economic impact. The total economic impact of the complexity change caused by a given project is then calculated as the sum of the cost changes per system, which is either given directly as a monetary value or as the product of the cost base and the percentage change for each system.

6. IMPLEMENTATION IN THE ORGANIZATION

IT management decided in August 2009 that new projects at Telenor Norway should be assessed with respect to the extent to which those projects will affect the overall IT complexity and the consequential costs. The results of this assessment should be included in each business case. It was also decided that the method described above should be introduced gradually. To begin

with, only the larger projects identified by the IT Architecture Department will be subject to this method.

It will be the responsibility either of the project manager or the chief architect of a new project to ensure that its effect on the complexity of the portfolio is assessed under the new method. This assessment, together with the high level design of the project, will thereafter be examined by the Architecture Control Board.

Because the method is based on human judgement (as opposed to automatically derived data), the competence of the people involved in it is critical; each of them must have a fair understanding of those factors that contribute to IT complexity in the given context. The necessary competence in how to apply the method must also be built up throughout all the parts of the organization involved in systems development. Other affected groups, such as the Project Coordination Department, the Finance Department and the Investment Office, as well as external consultants, must support the use of the method as much as possible. This is also in their own interest, because the outcome of the method may also be used in their own planning processes.

There were several factors that led to the approval of this scheme by IT management:

- Many stakeholders in the organisation were already aware of the problem of how best to manage the constantly increasing complexity of the portfolio and its economic consequences.
- Managers, system architects and other system experts from different development groups were involved in six pilot studies of the method. Their participation in the evaluation and refinement of the method has increased the confidence in, and commitment to, the implementation of the method in the organization. The method was also discussed with the local system architect interest group and external researchers.
- The concrete experiences obtained from the pilot studies indicate that we are on the right track. It makes sense; the method quantifies what is perceived to be right, although the method needs continuous improvement to become more accurate; using it is difficult, but so is conventional cost estimation. Despite this, using the method does not require significant effort. Nevertheless, a prerequisite to succeed is that the evaluation process is supported by a common architecture team that is well acquainted with the method and has the necessary experience to support the evaluation.
- The method and its planned use as part of individual business cases have been discussed with many interest groups in Telenor Norway, for example, the Project Coordination Department, Investment Office, the project managers and the most widely used external consultancy.
- This work is strategically important in that it complements other efforts in Telenor Norway to simplify and modernize the whole software portfolio.

7. FUTURE WORK AND LESSONS LEARNED

Although the overall experience of using the method is good, the method is still in an early stage and further validation and refinement is needed. It needs to be subject of a continuous improvement. During 2010, the method will be re-assessed and possibly revised following the recent experiences of using it. Its scope of application may also be reconsidered to include smaller

projects, which may in many cases make contributions of significant complexity. The checklists will be further refined and an experience database will be established. Some effort will be made to provide more accurate estimates of the cost bases of the various systems.

Telenor believes that by using the method, the organization will better understand the cost of IT complexity and will take this into account in project management. More specifically, we hope that this method will help to ensure that those projects that increase (or reduce) complexity are properly penalized (or rewarded) on the basis of the effect of any change in complexity in future projects. Those projects whose aim is to restructure, refactor, simplify, modernize, etc., and which are thus likely to reduce complexity, should in future enjoy a higher priority than they currently do, compared with ordinary business projects in which new functionality, interfaces, modules, etc. are generally developed, and which are thus likely to increase complexity overall. Furthermore, we hope that this method will promote the use of “good” architectures in projects, which in turn will have a positive impact on future projects.

Finally, we believe that the use of this method will result in an increased awareness of the importance of IT complexity in all the relevant parts of Telenor, including its management.

REFERENCES

- [1] J. Koskinen, J.J. Ahonen, H. Sivula, T. Tilus, H. Lintinen, and I. Kankaanpää.: Software Modernization Decision Criteria – An Empirical Study. Proc. of the 9th Europ. Conf. Softw. Maint. and Reeng., IEEE., pp. 324–331 (2005)
- [2] E.B. Swanson, The dimensions of maintenance, Proceedings of the Second Int.'l Conference on Software Engineering, San Francisco, USA, pp. 492–497 (1976)
- [3] S.L. Schneberger and E.R. McLean.: The Complexity Cross: Implications for Practice, Communications of the ACM, vol. 46, no. 9, pp. 216–225 (2003)
- [4] S.R. Chidamber and C.F. Kemerer.: A Metrics Suite for Object Oriented Design, IEEE Trans. Software Eng., vol. 20, no. 6, pp. 476–493 (1994)
- [5] B.C.D. Anda, D.I.K. Sjøberg, and A. Mockus.: Variability and Reproducibility in Software Engineering: A Study of Four Companies that Developed the Same System, IEEE Trans. Software Eng., vol. 35, no. 3, pp. 407–429 (2009)
- [6] P.W. Oman and J.R. Hagemester.: Construction and Testing of Polynomials Predicting Software Maintainability, J. Systems and Software, vol. 24, no. 3, pp. 251–266 (1994)
- [7] D. Darcy and C.F. Kemerer.: OO Metrics in Practice, IEEE Software, vol. 22, no. 6, Nov./Dec. 2005, pp. 17–19 (2005)
- [8] S. Chulani, P. Santhanam, B. Hodges, and K.B. Anders.: Metrics-Based Management of Software Product Portfolios, IEEE Software, vol. 24, no. 2, pp. 66–72 (2007)
- [9] M.M. Saarelainen, J. Koskinen, J.J. Ahonen, I. Kankaanpää, H. Sivula, H. Lintinen, P. Juutilainen, and T. Tilus.: Group Decision-Making Processes in Industrial Software Evolution, Int.'l Conf. Software Eng. Advances, pp. 78 (2007)
- [10] K.J. Moløkken-Østfold and M. Jørgensen.: Group Processes in Software Effort Estimation, Empirical Software Eng., vol. 9, no. 4, pp. 315–334 (2004)